

SABR Equipped with AI Wings

Improving Option Pricing under the SABR Volatility Model
Using Artificial Neural Networks

Project 6 | Following Funahashi (2023)

Supervisor: Eric André | emlyon Business School



1

Introduction

Problem, motivation, and project workflow

Problem & Approach

The Challenge

SABR model

Industry standard for IR options — no closed-form implied vol

Hagan approximation

Fast (~4 μ s) but systematic overestimation, worst in wings

Monte Carlo

Monte Carlo provides an accurate offline benchmark but is much slower than online inference.

Funahashi (2023) Approach

Residual learning — train ANN on error:

$$D = \sigma_{MC} - \sigma_{Hagan}$$

Online correction:

$$\sigma_{corrected} = \sigma_{Hagan} + ANN(\xi)$$

Goal: MC accuracy at Hagan speed

1 Phase 1
Pricing Tools



2 Phase 2
MC Simulation



3 Phase 3
Dataset Gen



4 Phase 4
ANN Training



5 Phase 5
Evaluation

Scenario A: Standard SABR (positive rates) | Scenario B: Free-Boundary SABR (negative rates)



2

SABR Pricing Framework

Phase 1 — Black, Bachelier & Hagan approximation

2.1 Pricing Models & Hagan Approximation

Black Model

`black_call(F, K, T,  $\sigma$ )`
`black_iv(F, K, T, price)`

Standard log-normal framework
Used for Scenario A implied vols

Bachelier Model

`bachelier_call(F, K, T,  $\sigma_N$ )`
`bachelier_iv(F, K, T, price)`

Normal model — allows $F < 0$
Used for Scenario B (negative rates)

Hagan SABR Approximation

`hagan_vol(F, K, T,  $\alpha$ ,  $\beta$ ,  $v$ ,  $\rho$ )`

Fast analytical baseline ($\sim 4 \mu s$)
Systematic overestimation in wings
→ ANN learns this residual error

SABR Model Dynamics

$$dF = v F^\beta dW_1 \quad dv = v v dW_2 \quad dW_1 dW_2 = \rho dt$$

Parameters: α (initial vol), β (CEV exponent), v (vol-of-vol), ρ (correlation)
No closed-form option price → Hagan approximation or Monte Carlo needed



3

Monte Carlo Benchmark

Phase 2 — Simulation engine & validation

3.1 Simulation Engine

Euler–Maruyama Discretisation

Standard SABR:

`simulate_sabr(F0,  $\alpha$ ,  $\beta$ ,  $v$ ,  $\rho$ , T, n_paths, n_steps)`

$F_{t+dt} = F_t + v_t F_t^\beta \sqrt{dt} Z_1$ Absorbing boundary at $F = 0$

Free-boundary SABR:

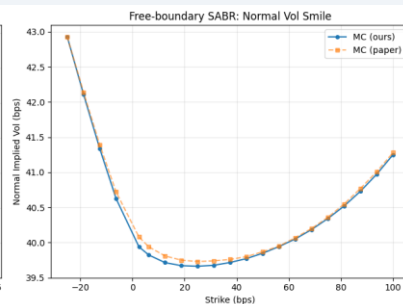
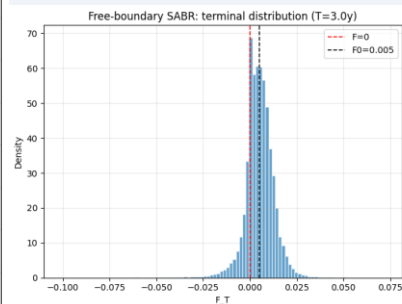
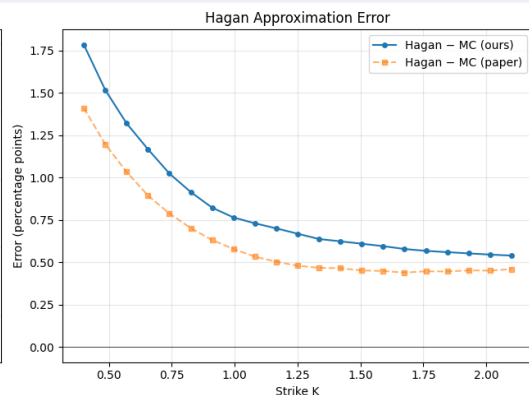
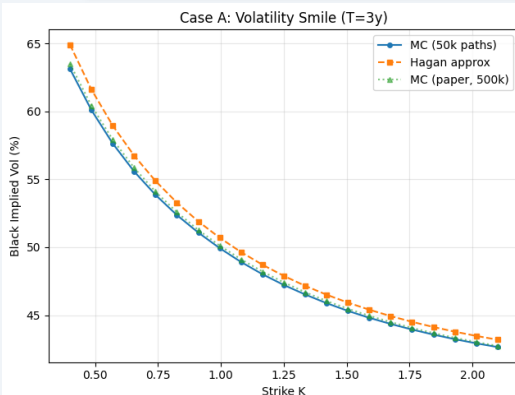
`simulate_sabr_free()` — uses $|F|^\beta$, allows $F < 0$ (negative rates)

Settings: 50k paths, 500 time steps, 4 maturities T = [1, 2, 3, 5]

MC Pricing Pipeline

1. Simulate terminal F_T under SABR dynamics
2. `mc_price()`: $E[\max(F_T - K, 0)]$
3. `mc_black_ivs()`: invert to Black implied vol
4. `mc_bachelier_ivs()`: invert to Normal vol (Sc. B)

σ_{MC} serves as ground truth benchmark



Left: MC smile vs Hagan vs published benchmark (Case A) Right: Free-boundary SABR terminal distribution & normal IV

3.2 Validation

Paper Replication

Funahashi Table 3 Cases A–D
Max diff < 1 bps across all strikes
All parameter sets match

Numerical Stability

$\beta \in [0.01, 0.99]$: finite outputs
 $\rho \in [-0.95, 0.95]$: stable
ATM continuity: smooth $K \rightarrow F$
Round-trip: < 1e-10 error

MC Engine Checks

Martingale: $E[F_T] \approx F_0$ (< 0.1%)
IV converges at 50k paths
Free-boundary: ~20% negative paths
Normal IVs match Table 12 range

All pricing tools and MC engines validated before dataset generation — benchmark correctness is critical



4

Dataset Construction

Phase 3 — Feature engineering & error analysis

Dataset Generation Pipeline

Random SABR Parameters

$\alpha, \beta, \nu, \rho \sim \text{Uniform}(\text{ranges})$



Monte Carlo Simulation

50k paths $\times$ 500 steps $\rightarrow F_T$



MC Implied Vol (σ_{MC})

$\text{mc_price}() \rightarrow \text{black_iv}() / \text{bachelier_iv}()$



Hagan Approximation (σ_{Hagan})

$\text{hagan_vol}()$ — fast analytical formula



Residual Error

$D = \sigma_{MC} - \sigma_{Hagan}$ (per strike, per maturity)



ANN Training Set

Scenario A: $X = [\alpha, \beta, \nu, \rho, T, \log(K/F)]$

Scenario B: $X = [\alpha, \beta, \nu, \rho, T, \log((K+s)/(F+s)), s]$

Scenario A

$M = 1000$ param sets

$\times 4$ maturities $\times 21$ strikes

$\approx 83k$ samples

$F = 1.0, K/F \in [0.5, 2.0]$

Scenario B

$M = 1000 \times 3$ shifts

$\times 4$ maturities $\times 21$ strikes

$\approx 236k$ samples

$F = 0.005, \text{shifts} = [50, 100, 200 \text{ bps}]$

Dataset generation runtime ($M=1000$, 4 maturities): ~ 88 min for Scenario A and ~ 85 min for Scenario B

4.1 Scenario A — Standard SABR

Features & Target

Target: $D = \sigma_{MC} - \sigma_{Hagan}$

Feature	Range
α (initial vol)	[0.1, 0.6]
β (CEV exponent)	[0.1, 0.9]
v (vol-of-vol)	[0.1, 0.5]
ρ (correlation)	[-0.7, 0.2]
T (maturity)	{1, 2, 3, 5}
$\log(K/F)$ (moneyness)	21 strikes

Dataset Statistics

M = 1000 parameter sets

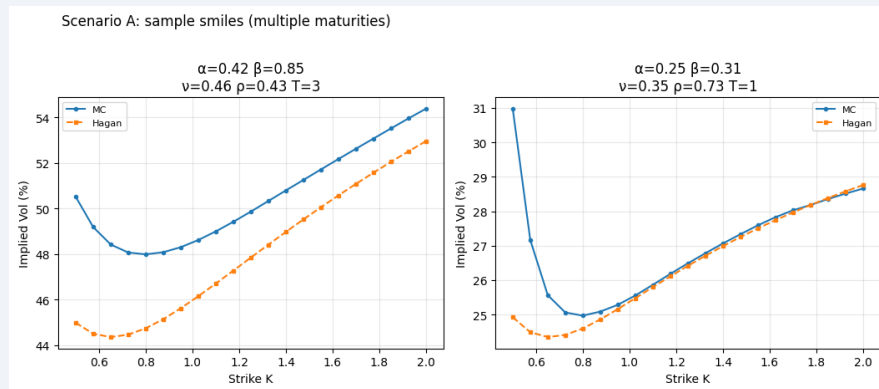
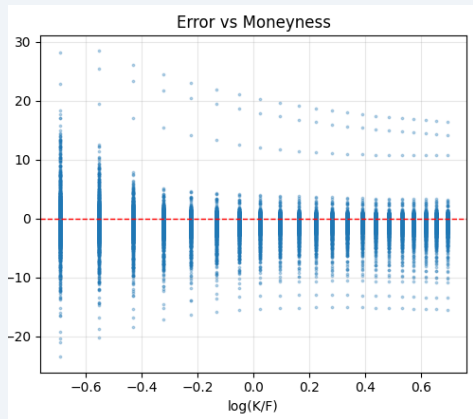
× 4 maturities × 21 strikes ≈ 83k samples

Error distribution:

Mean: -0.28 pp | Std: 0.90 pp

Min: -7.88 pp | Max: +14.93 pp

Negative mean → Hagan overestimates on average



4.2 Scenario B — Free-Boundary SABR (Negative Rates)

Scenario B Setup

$F = 0.005$ (50 bps), $\beta = 0.15$, $T = [1, 2, 3, 5]$

Shifts: $s \in \{50, 100, 200 \text{ bps}\}$

Feature: $\log((K+s)/(F+s))$ + shift as extra input

Baseline: shifted Hagan approximation

Target: $D = \sigma_{MC}(\text{Normal}) - \sigma_{App}(\text{shifted Hagan})$

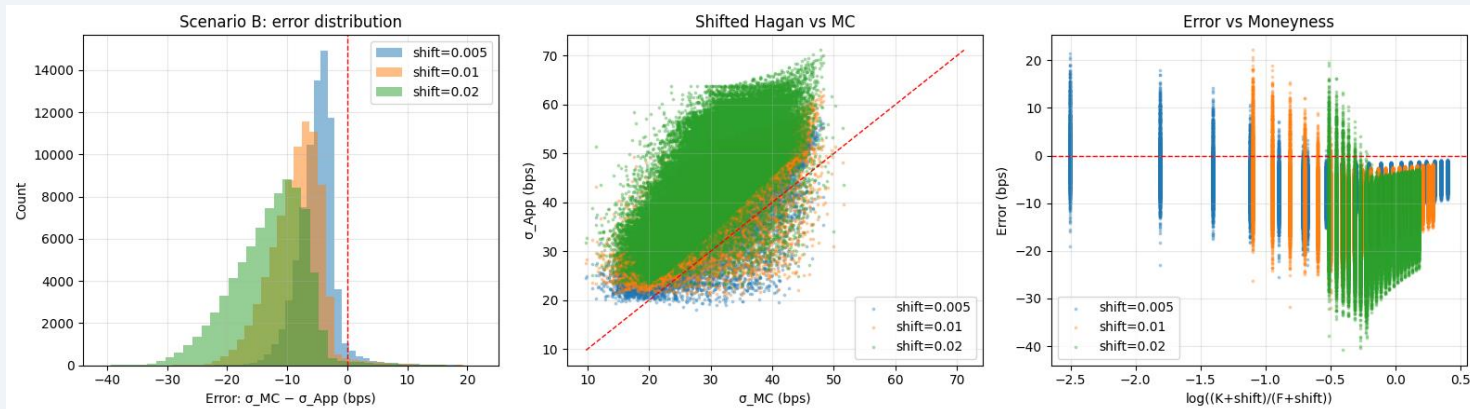
Dataset Statistics

$M = 1000 \times 3 \text{ shifts} \times 4 \text{ maturities}$

$\approx 236k$ training samples

Error: mean -16.61 bps, std 8.79 bps

Shifted Hagan systematically overestimates



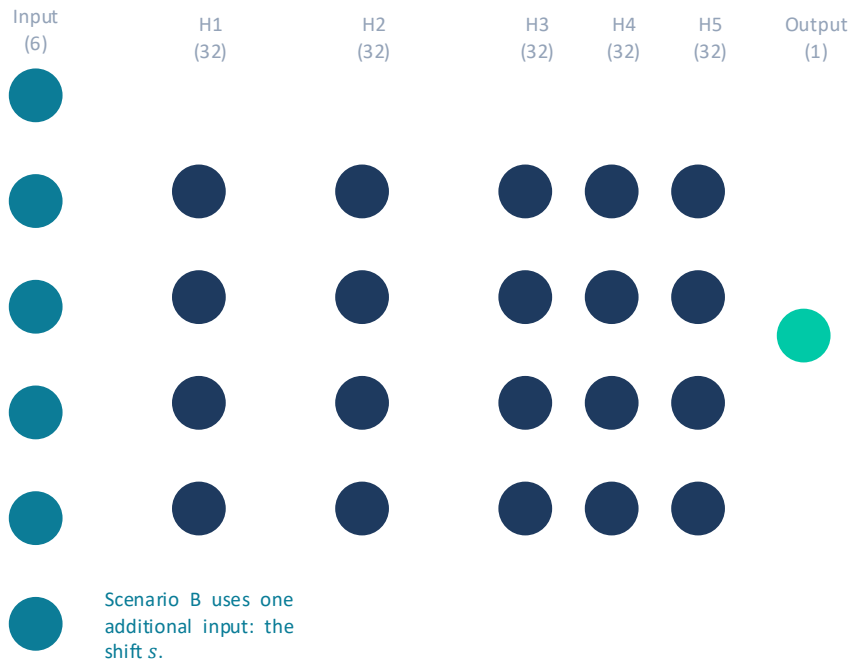


5

ANN Model

Phase 4 — Architecture & residual learning

5.1 Residual Learning & Network Architecture



Also tested: 3-layer variant (32,32,32)

Input Features (6)

alpha (vol-of-vol level)
beta (CEV exponent)
nu (vol-of-vol)
rho (correlation)
T (maturity)
log(K/F) (moneyness)

Target

$D = \sigma_{MC} - \sigma_{Hagan}$
(approximation error)

Training Setup

Activation: ReLU
Optimizer: Adam
Early stopping enabled
StandardScaler on X and y

6

Results

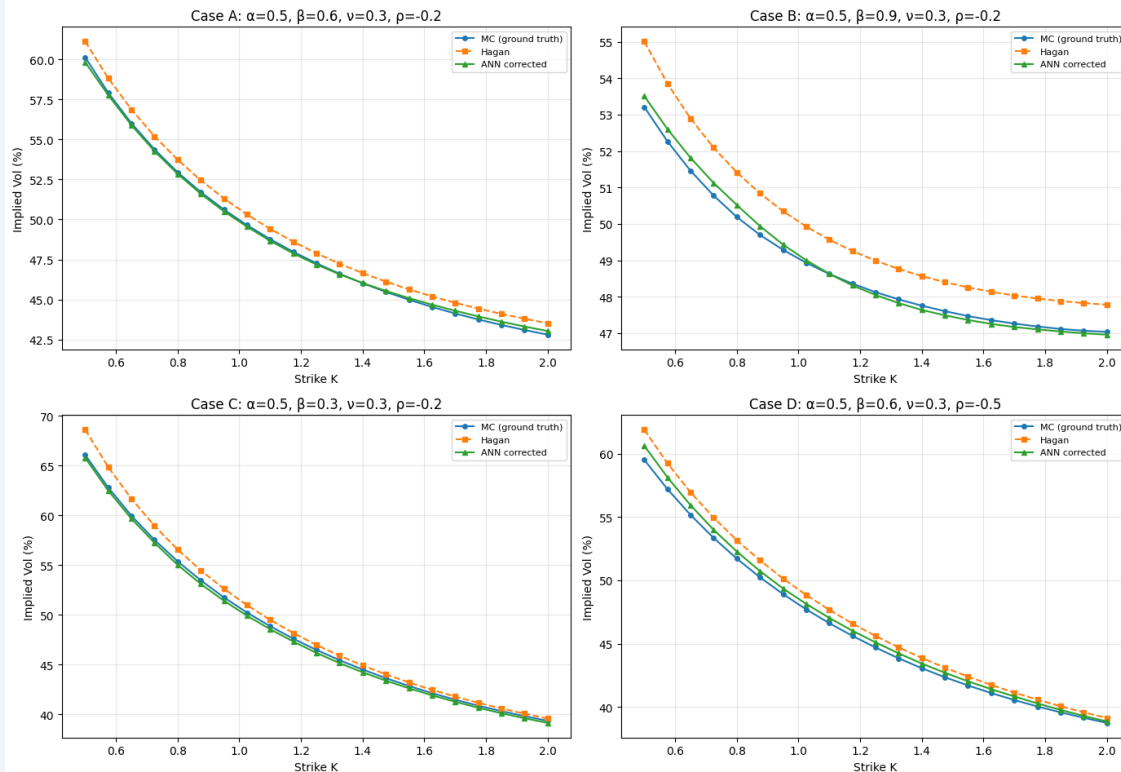
Phase 5 — Accuracy, error structure & speed

6.1 Volatility Smile Comparison

ANN corrects the systematic Hagan bias — corrected curve tracks MC benchmark closely

Cases A–D from Funahashi Table 2 | $T = 3$, $F = 1$, 50k MC paths

Scenario A: Volatility Smile Comparison ($T=3y$)



6.2 Scenario A — Accuracy

2.5×

RMSE Reduction

vs Hagan baseline

0.62 pp

ANN RMSE

on test set

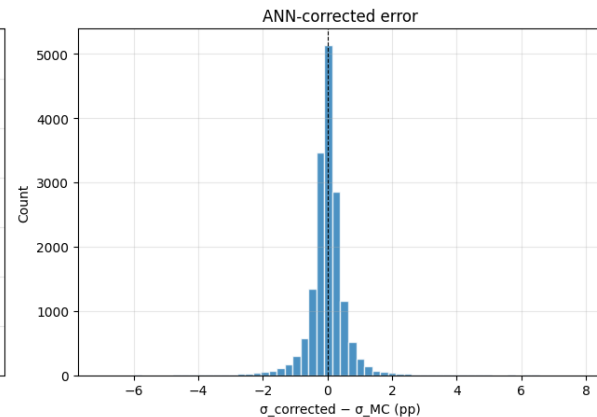
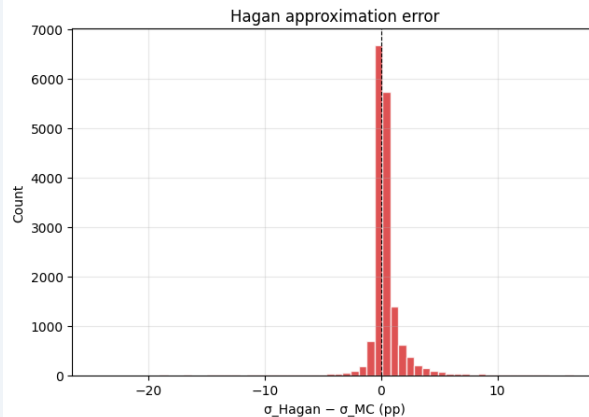
3.7×

Price Error ↓

92.98 → 25.13 bps

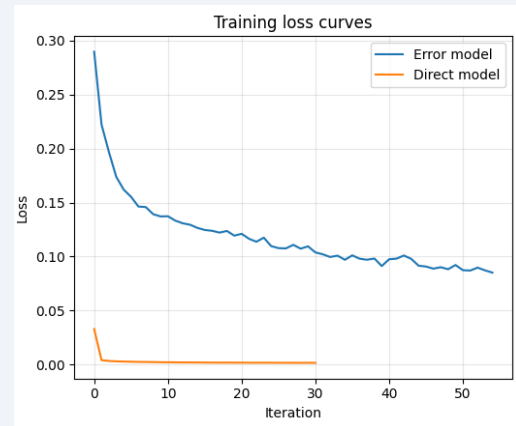
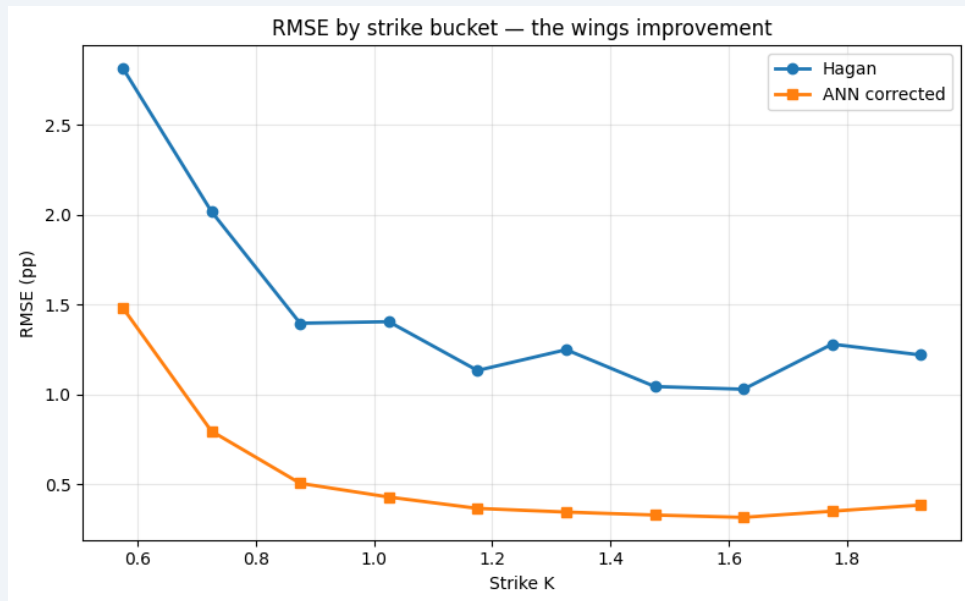
Method	RMSE (pp)	MAE (pp)	Max Err (pp)
Hagan (no correction)	1.5243	0.7239	24.4566
ANN Error Model	0.6159	0.3650	7.7816
ANN Direct Model	0.8330	0.5031	14.1048

Scenario A: Error Distribution (cf. Funahashi Fig. 2)



6.3 Error Structure — Wings Analysis

Hagan error is systematic and concentrated in the wings → learnable by ANN



6.4 Scenario B — Negative Rates Results

8.9×

RMSE Improvement

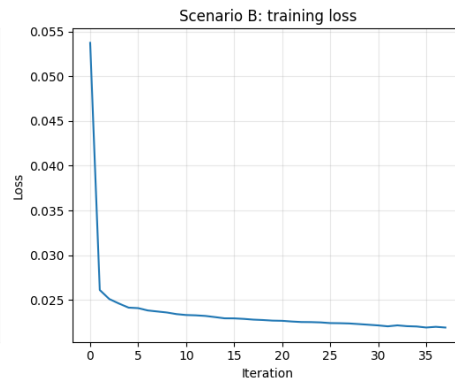
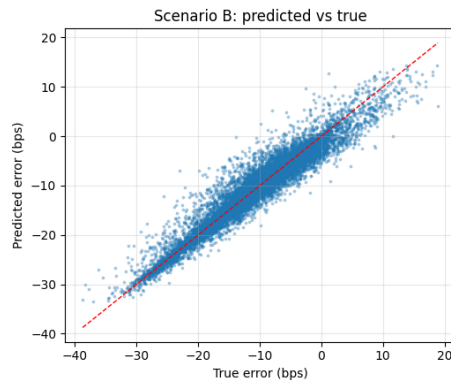
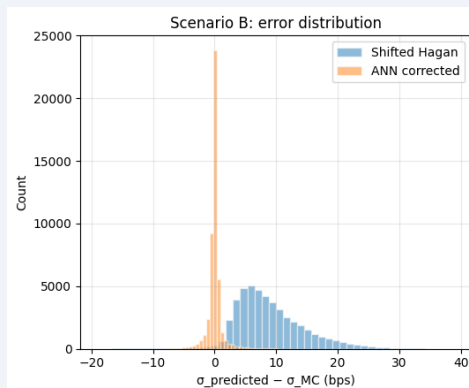
10.57 $\rightarrow$ 1.19 bps

1.19

ANN RMSE (bps)

on test set

Method	RMSE (bps)	MAE (bps)	Max Err (bps)
Shifted Hagan	10.57	9.11	38.76
ANN Error Model	1.19	0.62	16.02



Left: Scenario B error distributions (before/after ANN) Right: Scenario B detailed error analysis

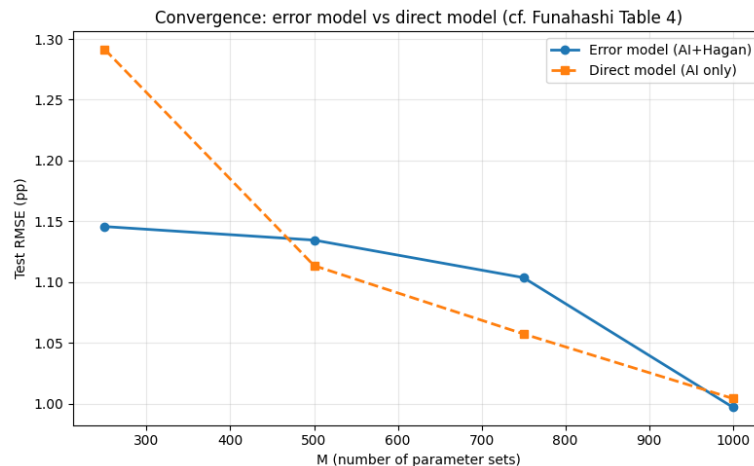
6.5 Architecture Experiment & Convergence

3 vs 5 Hidden Layers

Architecture	RMSE (pp)	Time (s)
3 layers (32,32,32)	0.5941	11.8
5 layers (32×5)	0.6159	9.0

*Depth has minimal effect — 3 layers marginally better.
Simpler architecture suffices for this dataset size.*

Convergence: Error vs Direct Model



6.6 Computational Cost: Offline vs Online

Offline Cost (one-time)

Paid once during model construction

Stage	Time	Samples
Dataset gen (Sc. A)	87.95 min	82,796
Dataset gen (Sc. B)	84.91 min	236,874
Convergence exp.	~218 min	207,020
ANN training (Sc. A)	~12 s	55 iters
ANN training (Sc. B)	~18 s	38 iters

*MC dataset generation is the dominant offline cost.
ANN training itself is relatively small (about 12–18 s).*

Online Inference (per prediction)

Paid every time you price an option

Method	Time	Usage
MC simulation (50k)	~1.5 s	benchmark
Hagan formula	4 μ s	baseline
ANN correction	1 μ s	batch avg
Hagan + ANN	5 μs	final

Timing interpretation

- MC refers to one benchmark simulation run
- Hagan / ANN refer to online inference latency
- ANN time is reported as batch-average per sample

The method is computationally expensive to build, but cheap to use.

Most of the cost is paid offline during MC dataset generation, while the final Hagan + ANN correction remains at the microsecond level for online pricing.



Discussion

Limitations & conclusions

7.1 Limitations

Finite Training Domain

ANN trained on specific parameter ranges — extrapolation outside these bounds may be unreliable

MC Label Noise

50k paths (vs 500k in paper) introduce noise in training labels, especially for deep OTM options

Euler–Maruyama Bias

Discretization bias near $F \approx 0$; exact simulation schemes could improve benchmark quality

Error Model Advantage

Learning the residual error converges faster and needs less data than learning σ_{MC} directly — confirmed experimentally

Key Takeaways

- ANN correction reduces Hagan RMSE by 2.5× (Scenario A) and 8.9× (Scenario B)
- Residual learning converges faster than direct σ_{MC} prediction — key Funahashi insight
- Error is systematic and strike-dependent — structured, not random noise
- 3 vs 5 hidden layers show similar accuracy — simpler architecture suffices
- Online prediction adds $\sim 1 \mu s$ — suitable for real-time pricing (300k× faster than MC)

References

Hagan et al. (2002) Managing Smile Risk

Antonov et al. (2013) SABR Spreads Its Wings

Funahashi (2023) SABR Equipped with AI Wings, Quantitative Finance 23(2)

Thank You

Questions?